

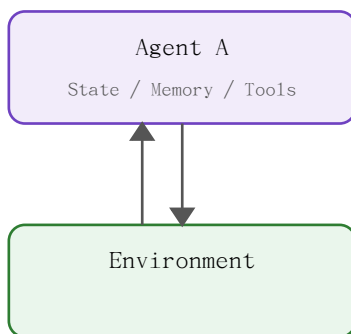
第29讲：单 Agent 与多 Agent

多个 Agent 到底多了什么？——角色、通信、共享状态、协调与集体决策

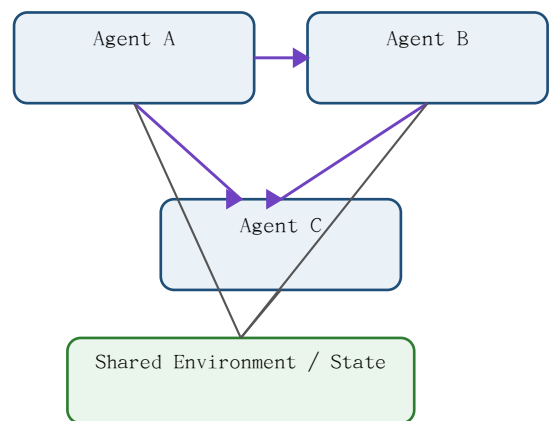
本讲目标

学完后你应该能解释：Multi-Agent System 不是“多开几个 LLM”；它真正增加的是多套状态/记忆、通信拓扑、协调、任务分配、共享环境和聚合机制。你还应理解为什么单体准确率相同，错误相关性不同会导致完全不同的集体收益。

单 Agent



多 Agent



多 Agent 多出来的不只是“数量”，还有通信、协调、共享状态和聚合。

先记住一句：多 Agent

的科学问题不在“多”，而在“交互之后出现了什么单体没有的系统行为”。

1. 多 Agent 不是“把同一个 Agent 复制 N 份”这么简单

如果只是把同一个模型、同一个 Prompt、同一个工具和同一个知识源重复调用 N 次，再做多数投票，这更接近 ensemble / repeated sampling。它可以有价值，但真正的 Multi-Agent System 往往还引入多套状态、角色、通信、共享或私有记忆、协调和聚合。

新增维度	关键问题
Multiple States	不同 Agent 是否知道不同的信息？
Roles	不同 Agent 负责什么？
Communication	谁能和谁交换什么消息？
Shared / Private Memory	哪些经历共享，哪些保留个体差异？
Coordination	怎样分工、同步、解决冲突？
Aggregation	怎样形成最终集体输出？
Shared Environment	一个 Agent 的动作是否改变其他 Agent 的世界？
Resource Allocation	总预算如何分配？

2. Role: 第一个新增变量

Role 是对 Agent 的功能、权限、信息或目标的约束。Prompt 角色、工具角色、信息角色、目标角色和权限角色都可能形成不同分工。

Role 形式	例子	可能形成的差异
Prompt Role	Planner / Critic / Reviewer	关注点和输出格式
Tool Role	搜索者 / 代码执行者	可执行动作不同
Information Role	各自看到不同文档	知识状态不同
Goal Role	成本优化 / 准确率优化	局部目标不同
Authority Role	提议者 / 审批者	决策权不同

科研提醒
角色标签本身不等于功能性异质性。真正要看的是角色是否让 Agent 在长期行为、工具使用、信息状态和错误模式上形成稳定差异。

3. Communication: 多个 Agent 怎样互相“看见”？

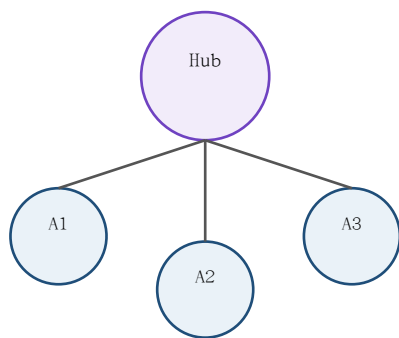
最简单的通信，就是把 Agent A 的输出追加到 Agent B 的 Context。但系统变大后，通信会变成独立研究变量。

变量	问题
Who	谁可以给谁发消息？
When	每步都发还是事件触发？

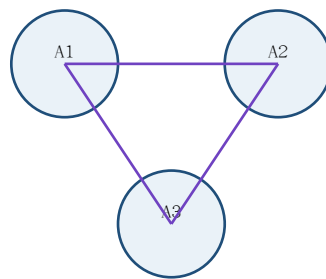
变量	问题
What	全文、摘要、证据还是结构化状态？
How much	消息 token 预算是多少？
Trust	接收到的信息是否直接相信？
Persistence	消息是否写入共享 Memory？

4. Communication Topology: 消息走哪条路？

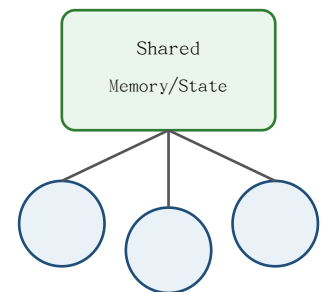
Hub / Coordinator



Peer-to-Peer



Shared Blackboard



Topology 决定消息路径、瓶颈、延迟，也会影响错误传播方式。

拓扑	优点	主要风险
Central Coordinator	控制清晰、易分工	单点瓶颈和单点错误
Peer-to-Peer	信息路径灵活	消息爆炸、协调复杂
Shared Blackboard	异步共享方便	错误写入后全局传播
Hierarchy	适合扩展大团队	上层汇总可能丢细节

系统视角

Topology 会改变消息延迟、信息扩散速度、错误传播范围和群体同步程度，因此不是纯工程细节。

5. Shared State 与 Private State

Shared State	Private State
所有 Agent 都能看到	只有个体 Agent 能看到
方便协调	保留独立视角
减少重复工作	可能形成知识不对称
错误可能快速同步	错误可能被局部隔离
容易导致同质化	更可能保留异质性

实际系统经常采用混合结构：任务进度和公共证据共享，而局部草稿、私有经历和某些专长信息保留在个体内部。

6. Shared Memory 与 Private Memory: 长期异质性的关键

如果所有 Agent 每轮都把全部经历同步到同一个共享 Memory，它们长期会越来越相似：

Full sharing -> Information synchronization -> Behavior convergence

如果保留不同经历，则可能形成：

Different experience -> Different memory -> Different retrieval -> Different future behavior

与你的研究直接相关
长期经历形成的功能性异质性，很可能需要一定程度的 private memory 或 asymmetric experience。全部共享可能提高短期效率，却削弱长期差异。

7. Coordination: 多 Agent 真正难在怎样一起工作



Coordination = 分解 + 分工 + 同步 + 冲突处理 + 汇总

如果这些步骤没有带来新信息或更低错误相关性，多 Agent 可能只是更贵。

最终要比较的是 Collective Gain / Compute，而不是 Agent 数量本身。

环节	核心问题
Task Decomposition	任务能否拆成适合并行/分工的子任务？
Allocation	哪个 Agent 做哪个子任务？
Synchronization	什么时候等待、合并、继续？
Conflict Resolution	不同结论冲突时怎么办？
Aggregation	局部结果怎样形成最终决策？

8. Task Allocation: 人人都做一遍不等于协作

模式	例子	主要价值
Redundancy	5 个 Agent 解同一道题	降低低相关随机错误
Specialization	搜索/计算/审查分别完成	利用能力和工具互补

模式	例子	主要价值
Parallelization	100 篇论文分给 10 个 Agent	缩短墙钟时间
Debate / Critique	一个提出，一个反驳	暴露错误和隐含假设

9. Aggregation: 集体最后怎样拍板？

方式	特点	风险
Majority Vote	简单、无需中心 Judge	错误相关时会一起错
Weighted Vote	不同 Agent 权重不同	权重估计可能偏
Judge / Arbiter	由一个 Agent 选最终答案	Judge 成为瓶颈
Consensus	讨论到一致	可能形成 groupthink
Evidence Merge	先合并证据再决策	需要去重和冲突处理

核心
Aggregation 不能凭空创造正确性。它能利用的是单体之间已经存在的互补信息和错误差异。

10. Error Correlation: 多 Agent 最关键的变量之一

假设每个 Agent 单独准确率都是 65%。若它们错误近似独立，多数投票可能显著提高准确率；若它们总在同一批题上一起错，投票几乎无效。

Individual Accuracy alone is not enough.

真正需要关心的是错误重叠与依赖结构，例如 $P(A_i \text{ wrong AND } A_j \text{ wrong})$ 。

直觉
投票真正利用的是“我错的时候，你未必也错”。如果大家总一起错，人数增加主要只增加成本。

11. Diversity 不等于 Functional Heterogeneity

概念	最短解释
Answer Diversity	答案表面是否不同
Behavior Diversity	工具/计划/路径是否不同
Error Overlap	错误样本集合重叠多少
Error Correlation	错误事件是否一起发生
Functional Heterogeneity	长期能力、策略和信息状态是否稳定不同

多个 Agent 可以用不同措辞给出同一个错误，因此 Answer Diversity 很高并不代表 Error Correlation 很低。

12. Groupthink / Wrong Consensus: 通信也会放大错误

通信既传播正确信息，也传播错误。一个自信但错误的 Agent 可能成为锚点，其他 Agent 逐渐跟随，形成高度一致但错误的结论。

机制	解释
Anchoring	先出现的答案成为锚点
Authority Bias	某个角色被过度信任
Information Cascade	后续 Agent 跟随前面结论
Shared Judge Bias	所有 Agent 被同一 Judge 偏好塑造
Memory Synchronization	错误进入共享 Memory 后长期传播

科研价值
“通信越多越好”并不成立。通信强度本身可能同时提高信息共享和错误相关性。

13. Multi-Agent $\neq$ More Test-Time Compute

一个常见混淆是：1 Agent $\times$ 1 call 对比 5 Agents $\times$ 4 rounds = 20 calls。后者更强，可能只是用了更多模型调用。

Baseline	控制什么
Single-shot Single Agent	最简单基线
Single Agent Best-of-N	控制多个候选带来的收益
Single Agent with same verifier	控制 Judge / Verifier 收益
Compute-matched Single Agent	控制总 token / calls / time
Multi-Agent	测试交互机制的额外贡献

原则
真正的 Collective Gain 应尽量在相同总计算预算下比较，而不是只比较 Agent 数量。

14. 多 Agent 的成本从哪里来？

成本项	为什么增长
LLM Calls	Agent 数量 $\times$ 轮数
Input Tokens	通信和历史不断进入 Context
Output Tokens	多个 Agent 都在生成
Tool Calls	多个 Agent 可能并行使用工具
Communication Tokens	Agent-to-Agent 消息
Coordinator / Judge	额外聚合调用
Latency	依赖、等待和同步

因此系统至少应同时报告: success + calls + tokens + tool calls + communication + latency + cost。

15. 什么时候多 Agent 更可能有真实价值?

条件	原因
任务可分解	不同 Agent 可以并行/专门处理子任务
信息分散	不同 Agent 拥有互补证据
错误低相关	冗余或投票能够纠错
工具异质	不同 Agent 有不同外部能力
需要互审	独立 verifier 能暴露错误
长任务	分工可降低单一 Context 负担

反过来, 如果任务不可分、Agent 完全同质、错误高度相关, 多 Agent 很可能只是增加成本。

16. 什么时候强 Single Agent 反而更合适?

场景	原因
简单短任务	协调开销大于收益
强模型已经稳定完成	不需要额外通信
信息完全共享且无专业分工	多 Agent 容易重复劳动
低延迟要求	多轮讨论会增加时间
预算严格	多 Agent 调用成本高

不是价值判断
Single Agent 与 Multi-Agent 是不同系统设计, 应根据任务结构、风险与资源约束选择。

17. Multi-Agent Evaluation: 至少要测四层

层	指标示例
Individual	单 Agent 准确率、工具成功率
Interaction	消息数、通信 token、冲突率
Collective	最终成功率、Collective Gain
Dependence	Error Overlap / Correlation / Agreement
Efficiency	总 Calls / Tokens / Latency / Cost
Robustness	Agent 失效、错误消息、工具故障下表现

18. 一个适合长期研究的机制链

可以把一个长期研究问题拆成:

Experience Divergence -> Functional Heterogeneity -> Error Decorrelation -> Collective Gain	
箭头	需要验证什么
Experience -> Heterogeneity	经历差异是否稳定改变 Memory、策略、工具选择？
Heterogeneity -> Decorrelation	行为差异是否真的降低错误重叠？
Decorrelation -> Gain	低相关错误是否让聚合结果更好？
Gain under budget	控制总 compute 后收益是否仍存在？

科研洞察
 这比“多 Agent 比单 Agent 高 3%”更接近机制研究，因为它尝试解释系统为什么产生集体收益。

19. 小实验：同样单体准确率，不同错误相关性

材料包中的 Python 实验把每个 Agent 的边际准确率固定在约 65%，只改变共享错误概率。

组	共享错误强度	含义
Independent-ish	0.0	错误近似独立
Low shared errors	0.2	存在少量共同难题
Highly correlated	0.7	大量样本一起对或一起错

然后比较 1、3、5、9 个 Agent 做 Majority Vote 的准确率。

观察重点 近独立错误组随着 Agent 数量增加，集体准确率明显上升；高相关错误组提升小得多。
--

20. 这个实验为什么重要？

它说明集体表现不能只写成“单体准确率 + Agent 数量”的函数，还必须加入错误依赖结构和聚合机制：

$$\text{Collective Performance} = f(\text{Individual Quality}, \text{Dependence}, \text{Aggregation}, \text{Budget})$$

这也是为什么多强 Agent 和多弱 Agent 不能只比较平均准确率，还要看错误是否互补。

21. 常见误区汇总

误区	更准确的理解
多 Agent = 多次采样	真正 MAS 还包含状态、通信和协调
答案不同 = 功能异质	可能只是 sampling noise
Agent 越多越强	错误相关和协调成本可能抵消收益
讨论越多越好	可能产生 groupthink 和 token 膨胀
多数票一定可靠	共同错误时多数会稳定地错
Role Prompt 足够制造异质性	需要验证长期行为和错误结构

误区	更准确的理解
最终准确率高就说明协作好	可能只是用了更多 compute

22. 练习题

- 1) Multi-Agent System 相比单 Agent，系统层面新增了哪些组件？
- 2) 为什么 Role Prompt 不等于功能性异质性？
- 3) 通信拓扑为什么会影响错误传播？
- 4) Shared Memory 与 Private Memory 各有什么优缺点？
- 5) Redundancy 和 Specialization 有什么区别？
- 6) 为什么单体准确率相同，集体投票结果仍可能差很多？
- 7) 什么叫 Groupthink / Wrong Consensus？
- 8) 为什么 Multi-Agent 实验必须有 compute-matched baseline？
- 9) 如果你想研究“长期经历形成异质性”，至少要测哪几个中间变量？
- 10) 设计一个实验判断通信到底是在增加有用信息，还是只是在增加一致性。

答案见材料包《03_第29讲_练习题与答案.pdf》。

23. 本讲总结

一句话总纲 多 Agent 的价值不来自“数量”本身，而来自多套状态、信息、工具和经历通过通信与协调形成互补，并在错误不过度相关的条件下，经聚合产生超过相同预算单体系统的收益。	
组件	最短记法
Role	谁负责什么
Communication	谁和谁交换什么
Topology	消息走哪条路
Shared / Private State	什么共享，什么个体保留
Coordination	如何分工、同步、解决冲突
Aggregation	如何形成集体结论
Error Correlation	大家是不是一起错
Compute Matching	收益是不是只是花得更多
下一讲：第30讲《从 LLM 基础课走向 Agent Systems：完整知识地图与科研接口》。	